Digital Culture – Web Development with Django

Project Proposals

College Of Computing

Université Mohammed VI Polytechnique

Academic Year 2025-2026 | Semester 2

SplitTrip

Project 1: SplitTrip – Description & Context

Project Overview

SplitTrip is a collaborative expense management application for groups traveling together. It tracks individual expenses, calculates fair splits (equal, percentage-based, or custom), and generates optimized settlement plans that minimize the number of transactions needed to balance all debts within a group.

Real-World Context

- Group travel generates complex multi-party debts
- Manual tracking is error-prone and causes social friction
- Inspired by Splitwise, Tricount, and Settle Up
- Relevant to students, families, professional teams
- Addresses a universal need with algorithmic elegance

Target Users

- **Trip Organizer:** Creates groups, invites members, manages trip settings, exports reports
- **Group Member:** Adds expenses, views balances, settles debts, receives notifications
- **Guest (read-only):** Views shared summary without account (via invite link)

Project 1: SplitTrip – Features & Functionalities

Core Features

- ✓ **Trip Management:** CRUD trips with name, dates, description, cover image, currency
- ✓ **Member Management:** Invite by email, join via link, role assignment (organizer/member)
- ✓ **Expense Tracking:** Add expense with amount, payer, split type (equal/percentage/exact), category, receipt upload
- ✓ **Balance Calculation:** Real-time per-member balance with who-owes-whom matrix
- ✓ **Settlement Plan:** Optimized payment suggestions minimizing total transactions
- ✓ **Expense Categories:** Food, Transport, Accommodation, Activities, etc.

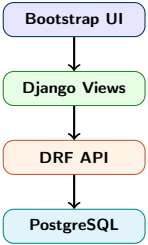
Project 1: SplitTrip – Technical Architecture

Django Models (Core Schema)

```python
class Trip(models.Model):
    name = CharField(max_length=200)
    creator = ForeignKey(User, on_delete=CASCADE)
    currency = CharField(default='MAD')
    start_date = DateField()
    end_date = DateField()

class Membership(models.Model):
    user = ForeignKey(User, on_delete=CASCADE)
    trip = ForeignKey(Trip, on_delete=CASCADE)
    role = CharField(choices=ROLE_CHOICES)

class Expense(models.Model):
    trip = ForeignKey(Trip, on_delete=CASCADE)
    paid_by = ForeignKey(User, on_delete=CASCADE)
    amount = DecimalField(max_digits=10,
        decimal_places=2)
    category = CharField(choices=CATEGORY_CHOICES)
    split_type = CharField(choices=SPLIT_CHOICES)

class ExpenseSplit(models.Model):
    expense = ForeignKey(Expense, on_delete=CASCADE)
    user = ForeignKey(User, on_delete=CASCADE)
    amount_owed = DecimalField(max_digits=10,
        decimal_places=2)
```

REST API Endpoints (DRF)

Method	Endpoint	Action
GET	/api/trips/	List trips
POST	/api/trips/	Create trip
POST	/api/trips/{id}/expenses/	Add expense
GET	/api/trips/{id}/balances/	Get balances
GET	/api/trips/{id}/settlements/	Settlement plan
POST	/api/trips/{id}/invite/	Invite member



Project 2 MazadLive — Auction Platform

Project 2: MazadLive – Description & Context

Project Overview

MazadLive is an online auction platform where users can list items for timed bidding, place competitive bids, and win items through transparent bidding processes. The system supports automated winner determination and notification.

Real-World Context

- Growing interest in second-hand economy in Morocco
- Complex state management: active, ended, won, paid
- Time-critical operations (bid windows, countdown)
- Combines e-commerce with competitive pricing logic

Target Users

- **Seller:** Lists items, sets starting price, reserve price, auction duration, manages shipping
- **Bidder:** Browses listings, places bids, sets auto-bid max, tracks active auctions
- **Admin:** Moderates listings, resolves disputes, manages categories

Project 2: MazadLive – Features & Architecture

Core Features

- ✓ **Listing Management:** Create auction with title, description, images, starting price, reserve price, end time
- ✓ **Bidding Engine:** Place bids with validation (higher than current, user not owner), auto-bid system
- ✓ **Auction Lifecycle:** Scheduled start/end, automatic winner determination, notification on win/outbid
- ✓ **Search & Filter:** By category, price range, time remaining, with sorting options
- ✓ **Watchlist:** Save auctions, get alerts before closing
- ✓ **Seller Dashboard:** Active listings, bid history, revenue analytics

Key Models

```
class Auction(models.Model):
    seller = ForeignKey(User, on_delete=CASCADE)
    title = CharField(max_length=300)
    description = TextField()
    starting_price = DecimalField(...)
    reserve_price = DecimalField(null=True)
    current_price = DecimalField(...)
    start_time = DateTimeField()
    end_time = DateTimeField()
    status = CharField(choices=STATUS_CHOICES)
    category = ForeignKey(Category, ...)
    winner = ForeignKey(User, null=True, ...)

class Bid(models.Model):
    auction = ForeignKey(Auction, ...)
    bidder = ForeignKey(User, ...)
    amount = DecimalField(...)
    timestamp = DateTimeField(auto_now_add=True)
    is_auto_bid = BooleanField(default=False)
```

REST API Design

Method	Endpoint	Action
GET	/api/auctions/	List active
POST	/api/auctions/	Create listing
GET	/api/auctions/{id}/	Detail + bids
POST	/api/auctions/{id}/bid/	Place bid
GET	/api/users/me/watchlist/	My watchlist
GET	/api/users/me/won/	Won auctions
GET	/api/categories/	Categories tree

Project 3 DARET – Rotating Savings Circle

Project 3: DARET – Description & Context

Project Overview

DARET digitizes the traditional Moroccan *Daret*. In a Daret, n members each contribute a fixed amount monthly; each month, one member receives the full pot. DARET automates member management, contribution tracking, payout scheduling, and trust verification with full transparency.

Cultural & Economic Context

- Estimated 40%+ of Moroccan households participate
- Trust-based informal finance with zero interest
- Common among diaspora communities worldwide
- Digital transformation enables transparency and dispute prevention

Target Users

- **Organizer (*Amin*):** Creates circle, sets rules, manages members, resolves conflicts, determines payout order
- **Member:** Joins circles, makes contributions, views schedule, receives payout, rates trust
- **Platform Admin:** Monitors active circles, handles escalated disputes, manages compliance

Project 3: DARET – Features & Functionalities

Core Features

- ✓ **Circle Creation:** Name, contribution amount, frequency (weekly/monthly), max members, rules
- ✓ **Member Enrollment:** Join requests, approval by organizer, invitation by link or email
- ✓ **Contribution Tracking:** Record payments per round, mark paid/pending/late, payment reminders
- ✓ **Payout Schedule:** Automatic or manual order (lottery, auction, seniority), calendar view
- ✓ **Circle Dashboard:** Current round, contribution status of all members, next payout recipient
- ✓ **History & Reports:** Full transaction log, member payment history, circle completion status

Django Models

```
class Circle(models.Model):
    name = CharField(max_length=200)
    organizer = ForeignKey(User, on_delete=CASCADE)
    contribution_amount = DecimalField(...)
    frequency = CharField(choices=FREQ_CHOICES)
    max_members = IntegerField()
    status = CharField(choices=['forming',
        'active', 'completed', 'cancelled'])
    current_round = IntegerField(default=0)
class CircleMembership(models.Model):
    circle = ForeignKey(Circle, on_delete=CASCADE)
    user = ForeignKey(User, on_delete=CASCADE)
    payout_order = IntegerField()
    trust_score = DecimalField(default=5.0)
class Contribution(models.Model):
    membership = ForeignKey(CircleMembership, ...)
    round_number = IntegerField()
    amount = DecimalField(...)
    status = CharField(choices=['paid','pending','late'])
    paid_date = DateTimeField(null=True)
class Payout(models.Model):
    circle = ForeignKey(Circle, on_delete=CASCADE)
    recipient = ForeignKey(User, on_delete=CASCADE)
    round_number = IntegerField()
    total_amount = DecimalField(...)
```

YourDoctorHere

project 4: YourDoctorHere – Description & Features

Project Overview

YourDoctorHere is a telemedicine appointment scheduling platform connecting patients with healthcare professionals. Doctors manage their availability slots, patients book consultations by specialty, and the system handles the full appointment lifecycle including confirmations, cancellations, medical notes, prescriptions, and consultation history.

Core Features

- ✓ **Doctor Profiles:** Specialty, qualifications, languages, consultation fee, bio, profile photo
- ✓ **Availability Management:** Doctors define weekly schedule slots (day, start/end time), block specific dates
- ✓ **Appointment Booking:** Patients search by specialty/city, view available slots, book with confirmation
- ✓ **Appointment Lifecycle:** Requested → Confirmed → Completed / Cancelled
- ✓ **Medical Notes:** Post-consultation notes by doctor (visible to patient), prescription text
- ✓ **Patient History:** Full consultation history per patient, accessible by both doctor and patient

Technical Highlights

- Slot generation algorithm from weekly schedule
- Search with multiple filters (specialty, city, date, fee range)
- Reminder system (management command for upcoming appointments)

Key Models

```python
class DoctorProfile(models.Model):
    user = OneToOneField(User, on_delete=CASCADE)
    specialty = ForeignKey(Specialty, ...)
    license_number = CharField(unique=True)
    consultation_fee = DecimalField(...)
    city = CharField(max_length=100)
    bio = TextField()
class WeeklySlot(models.Model):
    doctor = ForeignKey(DoctorProfile, ...)
    day_of_week = IntegerField()  # 0=Mon..6=Sun
    start_time = TimeField()
    end_time = TimeField()
class Appointment(models.Model):
    patient = ForeignKey(User, ...)
    doctor = ForeignKey(DoctorProfile, ...)
    date = DateField()
    time_slot = TimeField()
    status = CharField(choices=STATUS_CHOICES)
    reason = TextField()
class MedicalNote(models.Model):
    appointment = OneToOneField(Appointment, ...)
    diagnosis = TextField()
    prescription = TextField()
    created_at = DateTimeField(auto_now_add=True)
```

API Endpoints

Method	Endpoint	Action
GET	/api/doctors/	Search doctors
GET	/api/doctors/{id}/slots/	Available slots
POST	/api/appointments/	Book appointment
PATCH	/api/appointments/{id}/	Confirm/Cancel
POST	/api/appointments/{id}/notes/	Add notes
GET	/api/patients/me/history/	My history

Choose Your Project & Build Something Great!

Digital Culture: Web Development with Django

Projects – Academic Year 2025-2026

Coordinator: Pr. Fakir BENHLIMA

Platform: um6p.instructure.com (CANVAS)

Office Hours: By appointment